Distributed & Encrypted Storage System

Secure Cloud Architecture: A "Mini Dropbox/IPFS" Approach

Presented by Engineering Team Supervisor: J. Bruneau-Queyreix

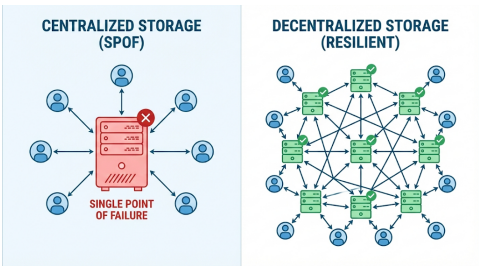
2026

Context: Data Privacy in the Cloud

The Problem:

- Centralized storage is a target for high-scale cyber-attacks.
- Structural vulnerabilities in traditional centralized architectures.
- Single Points of Failure (SPOF) lead to permanent data loss.



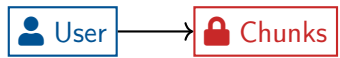


Decentralized System

Project Vision: Security by Design

Concept:

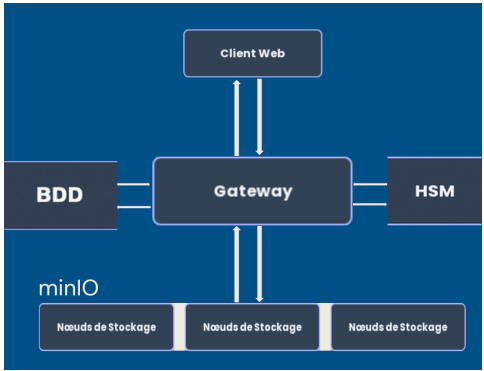
- Build a resilient "Mini Dropbox/IPFS".
- Zero-Trust approach via client-side encryption.
- Files encrypted, fragmented, and distributed.



Four Pillars of the Architecture

The System Components:

1. Web Interface (Client).
2. Central Service (Gateway).
3. Storage Nodes (MinIO Workers).
4. Metadata Database.

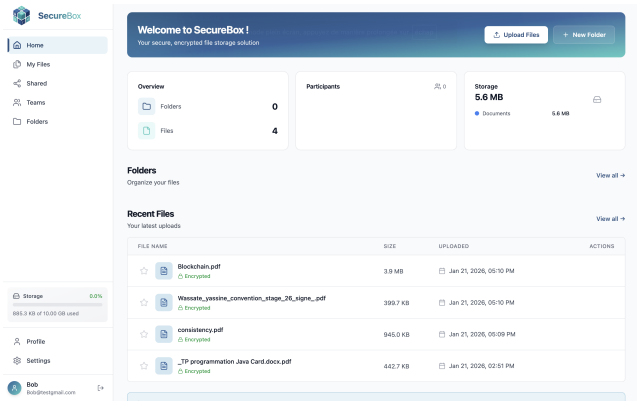


System Architecture Overview

Web Interface & Client-Side Logic

User Dashboard:

- </> Built with Vite, Tailwind CSS, and TypeScript.
- Secure authentication and session management.
- **E2EE:** All encryption and decryption are performed in-browser.
- Real-time feedback for upload and download actions.

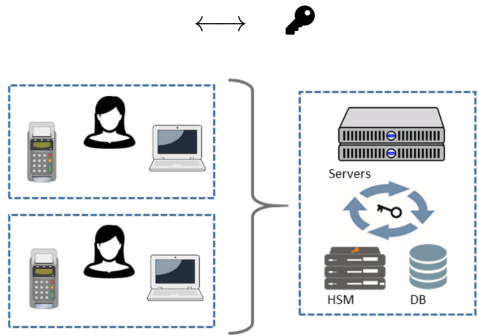


Integrated SecureBox Dashboard

Security: Virtual HSM Implementation

Key Management System:

- Integration of a **Virtual HSM** to secure cryptographic keys.
- Master keys are stored within the secure module boundaries.
- Sensitive cryptographic operations performed in a trusted environment.

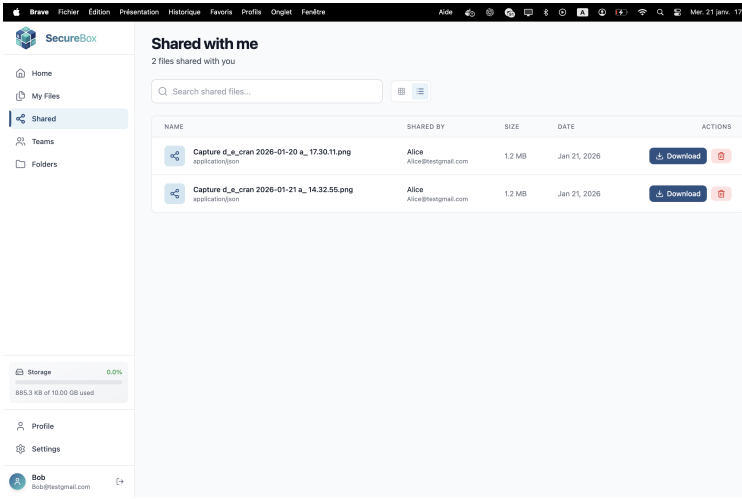


HSM method

File Sharing: Proxy Re-Encryption (PRE)

Secure Exchange Mechanism:

- **PRE mechanism** ensures confidentiality during sharing.
- Gateway re-encrypts data for recipients without seeing plaintext.
- Users retain sovereignty over their primary keys.

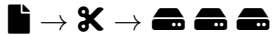


SecureBox Sharing Dashboard

Data Processing: Chunking

Distribution Strategy:

- Files fragmented into chunks for distributed storage.
- Chunks dispersed across independent **MinIO nodes**.
- Redundancy and error-correcting codes ensure data integrity.



Storage Tier: MinIO Object Storage

Why MinIO?

- **S3 Compatibility:** Standardized API for seamless integration with our Gateway.
- **Performance:** High-speed object storage optimized for large-scale chunk handling.
- **Open Source:** Flexible deployment via Docker containers for independent nodes.

System Operation:

- → Gateway pushes encrypted chunks via REST API.
- Each node manages its local persistence independently.



Screenshot: MinIO Console showing buckets and chunks

Resilience: Reed-Solomon Erasure Coding

Advanced Fault Tolerance:

- **The Principle:** A file is split into k data shards and m parity shards.
- **Recovery:** The original file can be reconstructed using any k shards out of $k + m$.
- **Efficiency:** High durability with much lower storage overhead than 3x replication.

Self-Healing:

- Detects corrupted or missing shards.
- Re-calculates and repairs data on-the-fly.

% INSERT IMAGE: Erasure Coding Diagram (Data + Parity Shards) %

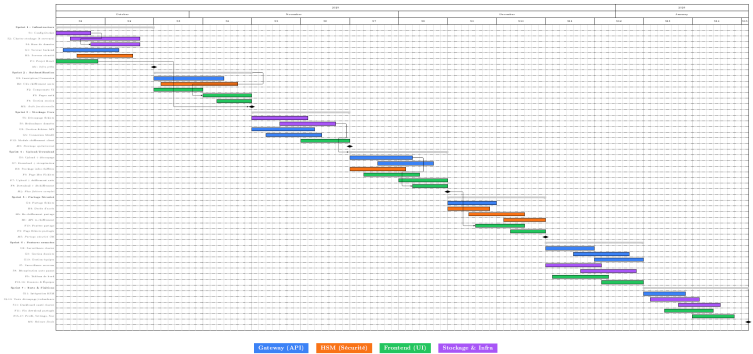
→

Survival even if multiple nodes fail simultaneously.

Management: Agile Methodology & Sprints

Iterative Development:

- Implementation of Agile development cycles.
- Task monitoring and coordination via **Jira**.
- Sprints organized with clear technical objectives.

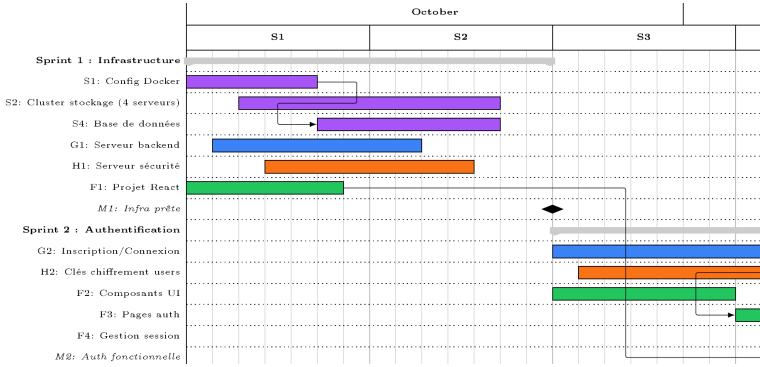


Gantt

Management: Focus on Core Development

🔍 Technical Sprints:

- Infrastructure Auth setup.
- Core Storage Chunking logic.
- Security HSM Integration.



Focus: First Configurations.

Management: Team Organization (RACI)

Responsibility Assignment:

- Clear task distribution.
- Accountability for security modules.
- Coordination for infra vs frontend.

Task / Responsibility	Richardson	Yassine H	Yassine W	Mayssen	Abdelaaziz	André	Kenzo
Final Report & Presentation	A	R	R	R	R	R	R
UI/UX Design & Frontend Dev	C	I	A	I	R	R	I
Client-side Encryption (AES-GCM)	C	I	R	I	R	A	I
Gateway API & Authentication	C	I	R	I	A	A	I
Virtual HSM & Sharing Logic	C	I	I	A	I	I	I
Metadata Management (PostgreSQL)	C	A	I	R	I	I	I
Erasure Coding & Parity Setup	A	I	I	I	I	I	R
Health Check & Node Monitoring	C	I	I	I	I	I	A
API REST Endpoints & Routing	C	A	I	R	I	I	I

R	A	C	I
Responsible	Accountable	Consulted	Informed

RACI Matrix: Roles Responsibilities

Management: Code Quality & Collaboration

Software Engineering Standards:

- **git** Source code management using **Git** for parallel development.
- Traceability of changes and technical decisions.
- >_ Regular meetings to resolve technical issues and coordinate.



% INSERT SCREENSHOT: Git commit graph or CI/CD pipeline %

Management: Technical Risks Triage

Risk Mitigation:

- Identifying Single Points of Failure in architecture.
- Continuous monitoring of node health via Gateway.
- Security tests: Alteration rejection and recovery.

Management Insight

Iterative progress allowed continuous validation of technical and security decisions.

Testing Performance Analysis

Validation:

- ✔ Comprehensive Unit and Integration tests.
- Security verification: Rejection of altered chunks.
- ⏱ Performance benchmarks: Timing vs File Size.

% INSERT SCREENSHOT: Performance Graph or Test report %

Live Testing Scenarios

- Upload Flow
- Download Flow
- Sharing Flow
- Broken node case



Switching to live demonstration...

Conclusion & Future Work

Success:

- Functional prototype of a secure distributed system.
- Mastery of distributed architectures and E2EE.

Future Steps:

- Self-healing algorithm optimization.
- High-concurrency stress testing.



Questions?